CS24307 • MODULE 4

Module 4: Classical Planning & Bayesian Networks

Topics 23 to 29 • STRIPS / PDDL, Graphplan Mutexes, Probability & Bayes Nets

---

Module 4 Table of Contents (Topics 23 to 29)

Topic 23 to 25: Classical Planning, STRIPS & Graphplan

In **Classical Planning**, states and actions are represented using explicit propositional or first-order fluents. Under the **STRIPS / PDDL language**:

STRIPS Action Representation:

$$\text{Action}(\mathbf{a}, \text{Precond} : \mathbf{P}, \text{Effect} : \text{Add}(\mathbf{A}) \wedge \text{Delete}(\mathbf{D}))$$

$$\text{Result}(\mathbf{s}, \mathbf{a}) = (\mathbf{s} \setminus \mathbf{D}) \cup \mathbf{A}$$

STRIPS Blocks World Problem Formulation

```
Init(On(A, Table) ∧ On(B, Table) ∧ Clear(A) ∧ Clear(B) ∧ ArmEmpty)
Goal(On(A, B) ∧ On(B, Table))

Action(PickUp(x),
  PRECOND: Clear(x) ∧ On(x, Table) ∧ ArmEmpty
  EFFECT:  ¬On(x, Table) ∧ ¬Clear(x) ∧ ¬ArmEmpty ∧ Holding(x))

Action(PutDown(x),
  PRECOND: Holding(x)
  EFFECT:  On(x, Table) ∧ Clear(x) ∧ ArmEmpty ∧ ¬Holding(x))

Action(Stack(x, y),
  PRECOND: Holding(x) ∧ Clear(y)
  EFFECT:  On(x, y) ∧ Clear(x) ∧ ArmEmpty ∧ ¬Holding(x) ∧ ¬Clear(y))

Action(Unstack(x, y),
  PRECOND: On(x, y) ∧ Clear(x) ∧ ArmEmpty
  EFFECT:  Holding(x) ∧ Clear(y) ∧ ¬On(x, y) ∧ ¬Clear(x) ∧ ¬ArmEmpty)
```

🏛️ **Planning Graph Mutex (Mutual Exclusion) Conditions**

In Graphplan, two actions at level A_i are **MUTEX** if:

1. **Inconsistent Effects:** One action negates an effect of the other (A adds P, B deletes P).
2. **Interference:** One action deletes a precondition of the other (A deletes P, B requires P).
3. **Competing Needs:** Preconditions of A and B are mutex at literal level S_i.

Topic 26 to 29: Probability, Bayes' Rule & Bayesian Networks

Bayes' Rule & Conditional Independence:

$$\mathbf{P(Y \mid X) = \frac{P(X \mid Y)P(Y)}{P(X)} = \frac{P(X \mid Y)P(Y)}{\sum_y P(X \mid Y = y)P(Y = y)}}$$

$$\mathbf{P(X, Y \mid Z) = P(X \mid Z) \cdot P(Y \mid Z)} \iff \mathbf{X} \text{ and } \mathbf{Y} \text{ conditionally independent given } \mathbf{Z}$$

🏛️ **Complete Step-by-Step Solved Problem: Earthquake-Burglary Bayesian Network Inference**

Given the classic Pearl Alarm network: B (Burglary), E (Earthquake), A (Alarm), J (JohnCalls), M (MaryCalls):

- $P(B = t) = 0.001, \quad P(E = t) = 0.002$
- $P(A = t \mid B = t, E = t) = 0.95,\ P(A = t \mid B = t, E = f) = 0.94,\ P(A = t \mid B = f, E = t) = 0.29,\ P(A = t \mid B = f, E = f) = 0.001$
- $P(J = t \mid A = t) = 0.90,\ P(J = t \mid A = f) = 0.05$
- $P(M = t \mid A = t) = 0.70,\ P(M = t \mid A = f) = 0.01$

Query: Calculate the exact joint probability that John calls and Mary calls, but there is NO burglary and NO earthquake, and the alarm did ring:

$$\mathbf{P(J = t, M = t, A = t, B = f, E = f) = P(B = f) \cdot P(E = f) \cdot P(A = t \mid B = f, E = f) \cdot P(J = t \mid A = t) \cdot P(M = t \mid A = t)}$$

$$= (0.999) \times (0.998) \times (0.001) \times (0.90) \times (0.70)$$

$$= \mathbf{0.997002 \times 0.001 \times 0.63 = 0.00062811 = 6.28 \times 10^{-4}}$$

Topic 29.2: Master University Examination Solved Question Bank (10 Solved Questions)

Q1. State and prove Bayes' Theorem from the Definition of Conditional Probability. (6 Marks)

By definition: $P(A \wedge B) = P(A \mid B)P(B)$ and $P(A \wedge B) = P(B \mid A)P(A)$. Equating the two expressions gives $P(A \mid B)P(B) = P(B \mid A)P(A)$. Dividing both sides by $P(B)$ yields: $\mathbf{P(A \mid B) = \frac{P(B|A)P(A)}{P(B)}}$.

Q2. Explain the difference between Progression Planning and Regression Planning. (8 Marks)

• **Progression Planning (Forward State-Space Search):** Starts from initial state S_0 and applies all applicable actions to generate successor states moving toward the goal. High branching factor on irrelevant actions.
• **Regression Planning (Backward Search):** Starts from the goal state G and applies inverted relevant actions backwards toward the initial state. Explores only actions that directly achieve a goal literal.

Q3. What is a Bayesian Network? How does it compress the Full Joint Probability Distribution? (8 Marks)

A Bayesian Network is a Directed Acyclic Graph (DAG) where nodes represent random variables and edges represent direct causal dependencies. For n binary variables, a Full Joint Distribution requires $2^n - 1$ parameters (exponential). A Bayes Net factors the joint distribution as: $\mathbf{P(X_1, \ldots, X_n) = \prod_{i=1}^{n} P(X_i \mid \text{Parents}(X_i))}$. If each node has at most k parents, it requires only $n \cdot 2^k$ parameters (linear in n!).

Q4. State the d-Separation rules for Causal Chains, Forks, and Colliders. (8 Marks)

1. **Causal Chain ($X \to Z \to Y$):** Blocked/Independent given Z.
2. **Common Cause / Fork ($X \leftarrow Z \to Y$):** Blocked/Independent given Z.
3. **Common Effect / Collider ($X \to Z \leftarrow Y$):** Independent if neither Z nor any descendant of Z is observed; **Becomes DEPENDENT (Active)** if Z or its descendant is observed! (Explaining Away phenomenon).

Q5. Explain the Graphplan Algorithm and why Planning Graphs can be computed in polynomial time. (8 Marks)

Graphplan constructs a Planning Graph of alternating state levels S_i and action levels A_i connected by precondition, add, delete, and persistence edges. Since actions are monotonic (literals only accumulate, never deleted from the graph level), the graph levels level off in polynomial time $O(n(a+l))$. Solution extraction is performed by backward search on mutex-free paths.

Q6. Explain the Sussman Anomaly in Classical Planning. (6 Marks)

The Sussman Anomaly is a classic failure mode of non-interleaved goal planning where achieving goal subgoal 1 ($On(A,B)$) undoes the prerequisite for subgoal 2 ($On(B,C)$), and vice-versa. It proves that simple decomposition of conjunctive goals without plan-step interleaving is sub-optimal and incomplete.

Q7. Detail the Variable Elimination Algorithm for Exact Inference in Bayesian Networks. (8 Marks)

Variable Elimination computes marginal distributions by dynamic programming:
1. Express query as sum of products of conditional probability table factors.
2. Choose an elimination ordering of non-query, non-evidence hidden variables.
3. Multiply all factors containing the variable to eliminate and sum out the variable.
4. Pointwise multiply the remaining factor and normalize by α.

Q8. Explain Markov Chain Monte Carlo (MCMC) and Gibbs Sampling in Bayesian Networks. (8 Marks)

When exact inference is NP-hard on dense networks, Gibbs Sampling generates approximate samples by:
1. Initializing all non-evidence variables randomly.
2. In each iteration, picking one non-evidence variable X_i and resampling its value conditioned on its Markov Blanket $P(X_i \mid \text{MB}(X_i))$.
3. Counting the frequencies of sampled states after a burn-in period to estimate posterior probabilities.

Q9. Explain Partial-Order Planning (POP) and contrast it with Total-Order Planning. (8 Marks)

Total-Order Planning strictly sequences actions into a single linear chain. **Partial-Order Planning** operates in the space of plans, establishing ordering constraints ($A \prec B$) and causal links ($A \xrightarrow{p} B$) only where necessary to resolve causal threats (demotions/promotions), leaving independent sub-plans unordered to facilitate parallel execution.

Q10. Calculate $P(\text{Cavity} \mid \text{Toothache})$ given: $P(\text{Toothache} \mid \text{Cavity}) = 0.8$, $P(\text{Cavity}) = 0.1$, and $P(\text{Toothache}) = 0.2$. (6 Marks)

Using Bayes' Rule:

$$\mathbf{P}(\text{Cavity} \mid \text{Toothache}) = \frac{\mathbf{P}(\text{Toothache} \mid \text{Cavity})\mathbf{P}(\text{Cavity})}{\mathbf{P}(\text{Toothache})} = \frac{(\mathbf{0.8})(\mathbf{0.1})}{\mathbf{0.2}} = \frac{\mathbf{0.08}}{\mathbf{0.20}} = \mathbf{0.40} = \mathbf{40\%}$$

Topic 29.5: Advanced Probabilistic Reasoning, HMMs & Kalman Filters

When reasoning over time in dynamic stochastic environments, agents use temporal probabilistic models to maintain probability distributions over unobserved hidden state variables.

Temporal Model	Mathematical State Representation	Filtering & Inference Task
Hidden Markov Model (HMM)	Discrete state space $S_t \in \{s_1, \dots, s_N\}$, discrete or continuous observations O_t. Transition matrix $T_{ij} = P(S_t = s_j \mid S_{t-1} = s_i)$, Emission matrix $E_{ik} = P(O_t = o_k \mid S_t = s_i)$.	Forward Algorithm (Filtering $P(S_t \mid o_{1:t})$), Backward Algorithm (Smoothing), Viterbi Algorithm (Most likely hidden path).
Kalman Filter (LGM)	Continuous state space with linear Gaussian transitions: $\mathbf{x}_t = \mathbf{F}\mathbf{x}_{t-1} + \mathbf{w}_t, \mathbf{z}_t = \mathbf{H}\mathbf{x}_t + \mathbf{v}_t$ (where $\mathbf{w}_t \sim \mathcal{N}(0, \mathbf{Q}), \mathbf{v}_t \sim \mathcal{N}(0, \mathbf{R})$).	Exact recursive update of Gaussian mean μ_t and covariance matrix $\mathbf{\Sigma}_t$.
Particle Filter (Sequential Monte Carlo)	Arbitrary non-linear, non-Gaussian continuous state space represented by a cloud of N weighted particles $\{(\mathbf{x}_t^{(i)}, w_t^{(i)})\}_{i=1}^N$.	Importance sampling, weight update by measurement likelihood, and resampling to prevent particle degeneracy.

🏛️ Step-by-Step Solved Problem: The Viterbi Dynamic Programming Algorithm for HMMs

Consider a 2-state weather HMM with states $\{\text{Rainy } (R), \text{Sunny } (S)\}$ and umbrella observations $\{U, \neg U\}$:

- Initial: $P(R_0) = 0.5,\ P(S_0) = 0.5$.
- Transitions: $P(R \mid R) = 0.7,\ P(S \mid R) = 0.3;\ P(R \mid S) = 0.3,\ P(S \mid S) = 0.7$.
- Emissions: $P(U \mid R) = 0.9,\ P(\neg U \mid R) = 0.1;\ P(U \mid S) = 0.2,\ P(\neg U \mid S) = 0.8$.

Observations: Day 1: U, Day 2: U. Find most likely state sequence:

Day 1 ($t = 1, O_1 = U$):

$$V_1(R) = P(R_0) \cdot P(U \mid R) = (0.5)(0.9) = \mathbf{0.45}$$

$$V_1(S) = P(S_0) \cdot P(U \mid S) = (0.5)(0.2) = \mathbf{0.10}$$

Day 2 ($t = 2, O_2 = U$):

$$V_2(R) = \max(V_1(R)P(R \mid R), V_1(S)P(R \mid S)) \cdot P(U \mid R) = \max((0.45)(0.7), (0.10)(0.3)) \cdot 0.9 = \max(0.315, 0.030) \cdot 0.9 = \mathbf{0.2835}$$

$$V_2(S) = \max(V_1(R)P(S \mid R), V_1(S)P(S \mid S)) \cdot P(U \mid S) = \max((0.45)(0.3), (0.10)(0.7)) \cdot 0.2 = \max(0.135, 0.070) \cdot 0.2 = \mathbf{0.0270}$$

$$\text{Most Probable Path: Day 1: Rainy} \rightarrow \text{Day 2: Rainy} \quad (\text{Joint Prob } = \mathbf{0.2835})$$

Q11. Explain the difference between Policy Iteration and Value Iteration in Markov Decision Processes. (8 Marks)

• **Value Iteration:** Iteratively updates value function $U_{k+1}(s)$ using the Bellman Optimality operator over all actions until convergence ($\|U_{k+1} - U_k\| < \epsilon$), then extracts the policy in a single final step. Can be slow because utility values take many iterations to converge to exact decimal precision.
• **Policy Iteration:** Alternates between two discrete phases: (1) *Policy Evaluation:* Calculates exact state utilities U^{π_k} for the current fixed policy π_k by solving a system of linear equations $O(N^3)$, and (2) *Policy Improvement:* Greedily updates the policy $\pi_{k+1}(s) = \arg\max_a \sum_{s'} P(s' \mid s, a)U^{\pi_k}(s')$. It converges in significantly fewer iterations because the policy space is finite ($|\mathcal{A}|^{|\mathcal{S}|}$).

Q12. What is Q-Learning? Why is it termed an Off-Policy Model-Free Algorithm? (8 Marks)

• **Model-Free:** Does NOT require explicit transition probability matrices $P(s' \mid s, a)$ or reward functions $R(s, a, s')$; it learns directly from sampled experiential trajectories $\langle s, a, r, s' \rangle$.
• **Off-Policy:** The target policy being learned is the *greedy optimal policy* ($\max_{a'} Q(s', a')$), which is completely decoupled from the *behavioral policy* (e.g. ϵ-greedy exploration) used to generate actions in the environment.

Topic 29.6: Advanced Decision Networks & Value of Information (VOI)

Value of Perfect Information (VPI / VOI): The expected value of information regarding an unobserved random variable E_j is the difference between the expected utility with E_j known versus without knowing E_j:

$$\text{VPI}(\mathbf{E_j}) = \left(\sum_{\mathbf{k}} \mathbf{P}(\mathbf{E_j} = \mathbf{e_{jk}}) \max_{\mathbf{a}} \mathbb{E}[\mathbf{U}(\mathbf{a} \mid \mathbf{e_{jk}})] \right) - \max_{\mathbf{a}} \mathbb{E}[\mathbf{U}(\mathbf{a})]$$

Properties of VPI: 1. Non-negative: $\text{VPI}(E_j) \geq 0$ (Information never reduces expected utility).
2. Non-additive: $\text{VPI}(E_j, E_k) \neq \text{VPI}(E_j) + \text{VPI}(E_k)$.
3. Order-independent: $\text{VPI}(E_j, E_k) = \text{VPI}(E_j) + \text{VPI}(E_k \mid E_j) = \text{VPI}(E_k) + \text{VPI}(E_j \mid E_k)$.

🏛️ Step-by-Step Solved Problem: Value of Information in Oil Drilling

An oil company is deciding whether to drill an offshore site (a_1) or sell the lease for $\$3\text{M}$ (a_2). Prior probability of oil is $P(\text{Oil}) = 0.5$. If oil is present, profit is $\$10\text{M}$; if dry, loss is $-\$4\text{M}$.

- Expected utility of drilling (a_1): $\mathbb{E}[U(a_1)] = 0.5(10) + 0.5(-4) = 5 - 2 = \mathbf{\$3}\text{M}$.
- Expected utility of selling (a_2): $\mathbb{E}[U(a_2)] = \mathbf{\$3}\text{M}$. Base optimal utility = $\$3\text{M}$.

A seismic survey (S) provides perfect information about whether oil is present:

- If seismic says Oil ($P = 0.5$): Drill (a_1) $\implies$ $\text{Utility} = \$10\text{M}$.
- If seismic says Dry ($P = 0.5$): Sell (a_2) $\implies$ $\text{Utility} = \$3\text{M}$.
- Expected utility with seismic test: $0.5(10) + 0.5(3) = 5 + 1.5 = \mathbf{\$6.5}\text{M}$.

$$\text{VPI}(\text{Seismic}) = \mathbf{\$6.5}\text{M} - \mathbf{\$3}\text{M} = \mathbf{\$3.5}\text{M}$$

Decision: The company should pay up to $\$3.5\text{M}$ for the seismic survey test!

Topic 29.7: Partially Observable MDPs (POMDPs) & Dynamic Decision Networks

When state transitions are stochastic and the state is only **partially observable** through noisy sensors, the agent operates in a **POMDP (Partially Observable Markov Decision Process)**:

The 7-Tuple POMDP Formalism:

$$\mathcal{P} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}(\mathbf{s}' \mid \mathbf{s}, \mathbf{a}), \mathcal{R}(\mathbf{s}, \mathbf{a}), \mathbf{\Omega}, \mathcal{O}(\mathbf{o} \mid \mathbf{s}', \mathbf{a}), \gamma \rangle$$

Where Ω is the set of observations, and $\mathcal{O}(o \mid s', a)$ is the observation emission probability. The agent maintains a **Belief State** $b(s)$ (a continuous probability distribution over all states $\sum_s b(s) = 1$).

🏛️ The Exact Belief State Update Equation (Bayesian Filtering)

After executing action a and perceiving observation o, the updated belief $b'(s')$ is computed via Bayes' Rule:

$$\mathbf{b}'(\mathbf{s}') = \frac{\mathcal{O}(\mathbf{o} \mid \mathbf{s}', \mathbf{a}) \sum_{\mathbf{s} \in \mathcal{S}} \mathcal{P}(\mathbf{s}' \mid \mathbf{s}, \mathbf{a}) \mathbf{b}(\mathbf{s})}{\mathbf{P}(\mathbf{o} \mid \mathbf{a}, \mathbf{b})} = \alpha \cdot \mathcal{O}(\mathbf{o} \mid \mathbf{s}', \mathbf{a}) \sum_{\mathbf{s} \in \mathcal{S}} \mathcal{P}(\mathbf{s}' \mid \mathbf{s}, \mathbf{a}) \mathbf{b}(\mathbf{s})$$

Interpretation: The continuous belief space $[0, 1]^{|\mathcal{S}|}$ converts a partially observable problem into a continuous, fully observable MDP over belief states!

🏛️ Step-by-Step Solved Problem: POMDP Belief State Update Calculation

A robot is in a 2-room corridor (S_1, S_2). Prior belief $b(S_1) = 0.5, b(S_2) = 0.5$. Action $a = \text{MoveRight}$:

- Transitions: $P(S_2 \mid S_1, \text{MoveRight}) = 0.8, P(S_1 \mid S_1, \text{MoveRight}) = 0.2;\ P(S_2 \mid S_2, \text{MoveRight}) = 0.9, P(S_1 \mid S_2, \text{MoveRight}) = 0.1$.
- Light sensor reading: $o = \text{Bright}$. Emissions: $P(\text{Bright} \mid S_1) = 0.1, P(\text{Bright} \mid S_2) = 0.8$.

Calculate updated belief $b'(S_1)$ and $b'(S_2)$:

$$\text{Predicted } P(S_1) = (0.2)(0.5) + (0.1)(0.5) = 0.10 + 0.05 = \mathbf{0.15}$$

$$\text{Predicted } P(S_2) = (0.8)(0.5) + (0.9)(0.5) = 0.40 + 0.45 = \mathbf{0.85}$$

$$b'(S_1) \propto P(\text{Bright} \mid S_1) \cdot 0.15 = (0.1)(0.15) = 0.015$$

$$b'(S_2) \propto P(\text{Bright} \mid S_2) \cdot 0.85 = (0.8)(0.85) = 0.680$$

$$\text{Normalization Constant } \alpha = \frac{1}{0.015 + 0.680} = \frac{1}{0.695} \approx 1.4388$$

$$\mathbf{b'(S_1) = 0.015 \times 1.4388 = \mathbf{0.0216 = 2.16\%} \qquad b'(S_2) = 0.680 \times 1.4388 = \mathbf{0.9784 = 97.84\%}}$$

Topic 29.8: Complete Step-by-Step Solved Problem Bank (Part III)

🏛️ Solved Numerical 3: Bayesian Network Exact Joint Probability Calculation

Consider a 4-node Bayesian Network with DAG structure $A \to B \to D$ and $A \to C \to D$:

- $P(A = t) = 0.4$.
- $P(B = t \mid A = t) = 0.8,\ P(B = t \mid A = f) = 0.2$.
- $P(C = t \mid A = t) = 0.7,\ P(C = t \mid A = f) = 0.1$.
- $P(D = t \mid B = t, C = t) = 0.9,\ P(D = t \mid B = t, C = f) = 0.6,\ P(D = t \mid B = f, C = t) = 0.5,\ P(D = t \mid B = f, C = f) = 0.1$.

Calculate $P(A = t, B = t, C = f, D = t)$:

$$\mathbf{P(A = t, B = t, C = f, D = t) = P(A = t) \cdot P(B = t \mid A = t) \cdot P(C = f \mid A = t) \cdot P(D = t \mid B = t, C = f)}$$

$$= (0.4) \times (0.8) \times (1 - 0.7) \times (0.6)$$

$$= (0.4) \times (0.8) \times (0.3) \times (0.6) = 0.32 \times 0.18 = \mathbf{0.0576 = 5.76\%}$$

🏛️ Step-by-Step Solved Problem: STRIPS Shakey the Robot World

Formulate the STRIPS operators for a mobile robot moving between rooms and pushing boxes:

```
Action(Go(x, y),
  PRECOND: At(Shakey, x) ∧ In(x, r) ∧ In(y, r)
  EFFECT:  ¬At(Shakey, x) ∧ At(Shakey, y))

Action(Push(b, x, y),
  PRECOND: At(Shakey, x) ∧ At(b, x) ∧ In(x, r) ∧ In(y, r) ∧ Box(b)
  EFFECT:  ¬At(Shakey, x) ∧ ¬At(b, x) ∧ At(Shakey, y) ∧ At(b, y))

Action(ClimbOn(b),
  PRECOND: At(Shakey, x) ∧ At(b, x) ∧ On(Shakey, Floor) ∧ Box(b)
  EFFECT:  ¬On(Shakey, Floor) ∧ On(Shakey, b))

Action(TurnOnLight(s),
  PRECOND: At(Shakey, x) ∧ At(s, x) ∧ On(Shakey, b) ∧ Box(b) ∧ Switch(s)
  EFFECT:  LightOn(s))
```

Q13. Explain Hierarchical Task Networks (HTN) Planning. (8 Marks)

HTN Planning extends classical planning by decomposing high-level compound abstract tasks (e.g. `TravelTo(NewYork)`) into smaller sub-tasks using **Methods** until only primitive actions remain (e.g., `BuyTicket`, `DriveToAirport`, `Fly`). HTN plans are dramatically faster to compute than STRIPS because domain-specific expert knowledge restricts the search space to realistic human-like decompositions rather than searching through arbitrary combinations of primitive actions!

Topic 29.9: Complete Partial-Order Planning (POP) Algorithm & Flaw Repair

🏛️ Step-by-Step Solved Problem: Partial-Order Planning (POP) Algorithm Walkthrough

Consider the classic **"Put on Shoes and Socks"** planning domain:

- Actions: $\text{RightSock}, \text{RightShoe}, \text{LeftSock}, \text{LeftShoe}$.
- Preconditions & Effects:
  - RightShoe: Precondition RightSockOn; Effect RightShoeOn.
  - LeftShoe: Precondition LeftSockOn; Effect LeftShoeOn.
- Goal: $\text{RightShoeOn} \wedge \text{LeftShoeOn}$.

POP Execution Steps:

1. Start with initial dummy plan: $\text{Start} \prec \text{Finish}$. Open preconditions: $\text{RightShoeOn}, \text{LeftShoeOn}$.
2. Achieve RightShoeOn by adding step RightShoe with causal link $\text{RightShoe} \xrightarrow{\text{RightShoeOn}} \text{Finish}$.
3. Achieve LeftShoeOn by adding step LeftShoe with causal link $\text{LeftShoe} \xrightarrow{\text{LeftShoeOn}} \text{Finish}$.
4. Resolve open precondition RightSockOn of RightShoe by adding RightSock with causal link $\text{RightSock} \xrightarrow{\text{RightSockOn}} \text{RightShoe}$.
5. Resolve open precondition LeftSockOn of LeftShoe by adding LeftSock with causal link $\text{LeftSock} \xrightarrow{\text{LeftSockOn}} \text{LeftShoe}$.
6. **Threat Checking:** No action deletes any established causal link ($\text{Threats} = \emptyset$).

$$\text{Final Partial Order Plan: } (\text{RightSock} \prec \text{RightShoe}) \wedge (\text{LeftSock} \prec \text{LeftShoe})$$

Power of POP: The left-foot and right-foot operations remain completely unordered with respect to each other, allowing parallel execution on a dual-arm robot!

🏛️ Complete Step-by-Step MCMC Gibbs Sampling Trace on Bayesian Networks

Given 3 variables C (Cloudy), R (Rain), W (WetGrass) with query $P(C \mid W = \text{true})$:

- Evidence: $W = \text{true}$. Hidden non-evidence variables: C, R.
- **Step 0:** Randomly initialize hidden variables: $C_0 = \text{true},\ R_0 = \text{false}$. State = $(\text{true}, \text{false}, \text{true})$.
- **Iteration 1 (Sample C):** Compute $P(C \mid R = \text{false}, W = \text{true}) = \alpha P(C) P(R = \text{false} \mid C) P(W = \text{true} \mid C, R = \text{false})$. Suppose distribution evaluates to $[0.35, 0.65]$. Sample random $u = 0.42 > 0.35 \implies C_1 = \text{false}$.
- **Iteration 2 (Sample R):** Compute $P(R \mid C = \text{false}, W = \text{true})$. Suppose evaluates to $[0.82, 0.18]$. Sample random $u = 0.10 < 0.82 \implies R_1 = \text{true}$.
- **Iteration 3 to N:** Repeat sampling. Tally the number of iterations where $C = \text{true}$ versus $C = \text{false}$ to compute the empirical posterior probability!

Topic 29.10: Master University Exam Problem Bank (Part IV)

🏛️ Solved Numerical 4: Exact Inference by Enumeration in 5-Node Bayesian Network

Given DAG $A \to B \to C$, with conditional probability tables:

- $P(A = t) = 0.3$.
- $P(B = t \mid A = t) = 0.9,\ P(B = t \mid A = f) = 0.1$.
- $P(C = t \mid B = t) = 0.8,\ P(C = t \mid B = f) = 0.2$.

Query: Compute posterior probability $P(A = t \mid C = t)$:

$$P(A = t, C = t) = P(A = t) \sum_b P(b \mid A = t) P(C = t \mid b) = 0.3[(0.9)(0.8) + (0.1)(0.2)] = 0.3[0.72 + 0.02] = 0.3(0.74) = \mathbf{0.222}$$

$$P(A = f, C = t) = P(A = f) \sum_b P(b \mid A = f) P(C = t \mid b) = 0.7[(0.1)(0.8) + (0.9)(0.2)] = 0.7[0.08 + 0.18] = 0.7(0.26) = \mathbf{0.182}$$

$$\mathbf{P(A = t \mid C = t) = \frac{0.222}{0.222 + 0.182} = \frac{0.222}{0.404} = 0.5495 = 54.95\%}$$

🏛️ Step-by-Step Solved Problem: STRIPS Tire Changing Domain

```
Init(Tire(Flat) ∧ Tire(Spare) ∧ At(Flat, Axle) ∧ At(Spare, Trunk))
Goal(At(Spare, Axle) ∧ At(Flat, Trunk))

Action(Remove(t, loc),
  PRECOND: At(t, loc)
  EFFECT:  ¬At(t, loc) ∧ Holding(t))

Action(PutOn(t, Axle),
  PRECOND: Holding(t) ∧ ¬At(Flat, Axle) ∧ ¬At(Spare, Axle)
  EFFECT:  At(t, Axle) ∧ ¬Holding(t))

Action(PutIn(t, Trunk),
  PRECOND: Holding(t)
  EFFECT:  At(t, Trunk) ∧ ¬Holding(t))
```

Q14. Explain Real-Time Heuristic Search (RTA* and LRTA*). (8 Marks)

In dynamic physical environments where planning time is strictly bounded, **Learning Real-Time A^* (LRTA*)** executes action moves within fixed time bounds by searching forward only a few steps, evaluating leaf nodes with heuristic $h(n)$, choosing the best immediate action, and updating its heuristic table at the visited state s: $\mathbf{h(s) \leftarrow \min_a [c(s, a, s') + h(s')]}$. Over repeated trials, LRTA* is guaranteed to converge to the optimal path!

Topic 29.11: Formal Decision Theory, Axioms of Utility & Influence Diagrams

Decision Theory unifies probability theory with utility theory: $\text{Decision Theory} = \text{Probability Theory} + \text{Utility Theory}$. An agent represents preferences between lotteries using scalar utility functions.

Axiom of Rationality (Von Neumann-Morgenstern)	Mathematical Formulation	Implication for Rational Agents
Orderability (Completeness)	$(A \succ B) \vee (B \succ A) \vee (A \sim B)$	An agent cannot avoid making a choice between any two states or lotteries.
Transitivity	$(A \succ B) \wedge (B \succ C) \implies (A \succ C)$	Prevents cyclical preferences (intransitive agents become "money pumps" exploited by opponents).
Continuity	$A \succ B \succ C \implies \exists p \in [0,1] \text{ s.t. } [p, A; (1-p), C] \sim B$	There is always a lottery between best and worst outcomes equivalent to an intermediate outcome.
Substitutability (Independence)	$A \sim B \implies [p, A; (1-p), C] \sim [p, B; (1-p), C]$	Indifference between two outcomes is preserved when embedded in larger compound lotteries.
Monotonicity	$A \succ B \wedge (p > q) \implies [p, A; (1-p), B] \succ [q, A; (1-q), B]$	Agents strictly prefer lotteries with higher probabilities of receiving the more desirable prize.

🏛️ Step-by-Step Solved Problem: Influence Diagram Evaluation Algorithm

An **Influence Diagram (Decision Network)** augments Bayesian Networks with *Decision Nodes* (rectangles) and *Utility Nodes* (diamonds):

1. **Evaluation Algorithm:**
   - For each possible assignment to the decision node $D = d_i$:
   - Set evidence $D = d_i$ in the network.
   - Calculate posterior probabilities $P(X \mid d_i)$ for parents X of the utility node U using Variable Elimination.
   - Compute expected utility: $\mathbb{E}[U \mid d_i] = \sum_x P(X = x \mid d_i) U(x, d_i)$.
2. **Optimal Policy:** Choose action $d^* = \arg\max_{d_i} \mathbb{E}[U \mid d_i]$!

🏛️ The St. Petersburg Paradox & Risk Neutrality vs. Risk Aversion

A fair coin is tossed until heads appears on flip n. The player receives prize $\$2^n$. What is the fair price to play?

- **Expected Monetary Value (EMV):** $\mathbb{E}[\text{Money}] = \sum_{n=1}^{\infty} \left(\frac{1}{2}\right)^n (2^n) = \sum_{n=1}^{\infty} 1 = \mathbf{1} + \mathbf{1} + \mathbf{1} + \cdots = \boldsymbol{\infty}$.
- Yet real humans will only pay $\sim \$10$ to $\$25$ to play this game!
- **Bernoulli's Resolution (1738):** Humans maximize *Expected Utility* $U(w) = \ln(w)$ (concave logarithmic utility function), NOT raw monetary payout!

$$\mathbb{E}[U] = \sum_{n=1}^{\infty} \left(\frac{1}{2}\right)^n \ln(2^n) = \ln(2) \sum_{n=1}^{\infty} \frac{n}{2^n} = \ln(2) \times 2 = \mathbf{2}\ln(\mathbf{2}) \approx \mathbf{1.386} \text{ utils}$$

$$U^{-1}(1.386) = e^{1.386} = \mathbf{\$4.00} \text{ (Certainty Equivalent)}$$

🏛️ Step-by-Step Solved Problem: Policy Iteration Dynamic Programming Trace on 3-State Chain

Consider a 3-state chain MDP with states $\{S_1, S_2, S_3\}$, actions $\{\text{Left}, \text{Right}\}$, discount $\gamma = 0.8$, and terminal absorbing state S_3 with reward $R(S_3) = +10$. Other step rewards are 0.

1. Initial Policy π_0: Always choose Right ($\pi_0(S_1) = \text{Right}, \pi_0(S_2) = \text{Right}$).

2. Policy Evaluation: Solve linear system $U^{\pi_0}(s) = R(s) + \gamma \sum_{s'} P(s' \mid s, \pi_0(s)) U^{\pi_0}(s')$:

$$U(S_2) = 0 + 0.8 U(S_3) = 0.8(10) = \mathbf{8.0}$$

$$U(S_1) = 0 + 0.8 U(S_2) = 0.8(8.0) = \mathbf{6.4}$$

3. Policy Improvement: Calculate $Q(s, a)$ for alternative action Left:

$$Q(S_1, \text{Left}) = 0 + 0.8 U(S_1) = 0.8(6.4) = 5.12 < 6.4 \implies \text{Keep Right}$$

$$Q(S_2, \text{Left}) = 0 + 0.8 U(S_1) = 0.8(6.4) = 5.12 < 8.0 \implies \text{Keep Right}$$

$$\text{Policy Converged: } \pi^*(\mathbf{S_1}) = \text{Right},\ \pi^*(\mathbf{S_2}) = \text{Right} \quad (\text{Optimal Policy Identified in 1 Iteration!})$$